

C PROGRAMMING - DEBUGGING TOOLS

Practical Assignment Report

Name: Dhruv Pandey

SAP ID: 590037660

Course: B.Tech CSE - 1st Year

Batch: 15

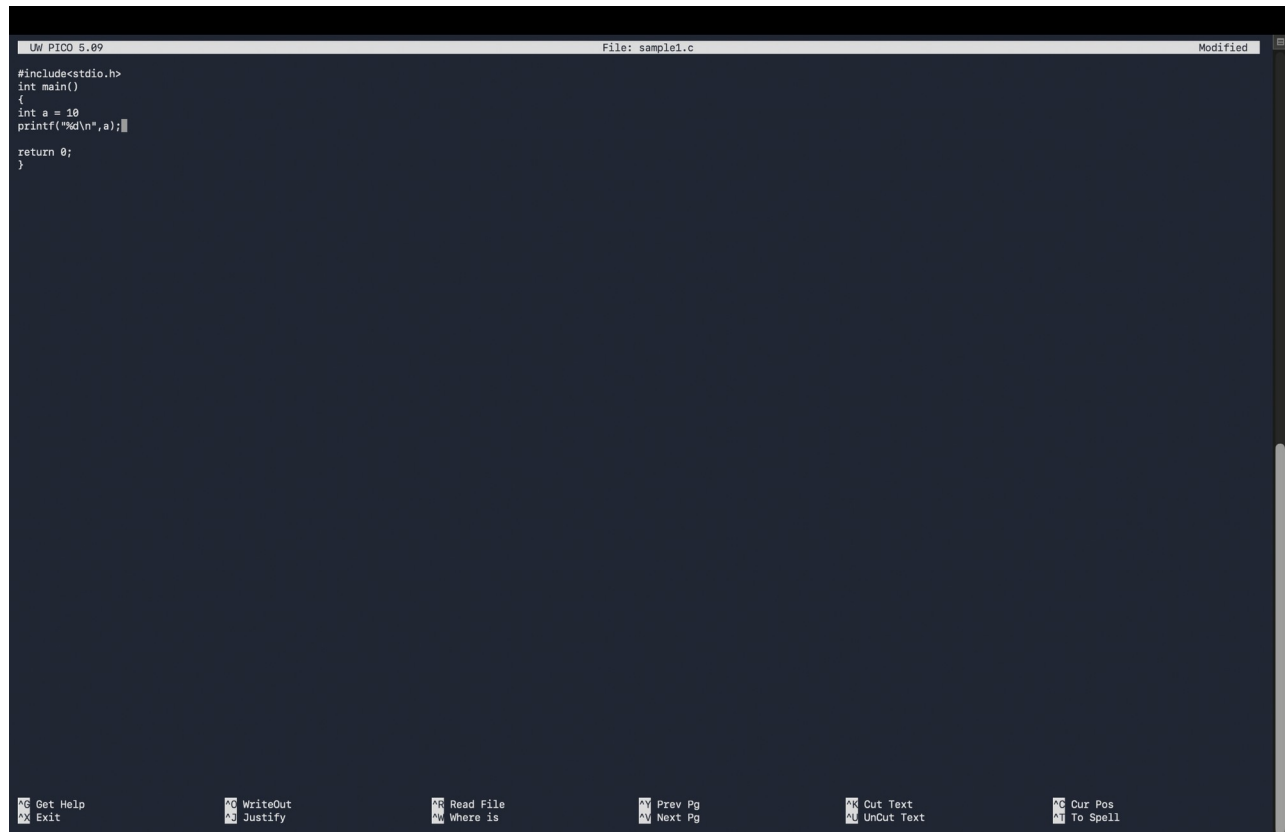
1. Fundamentals of Errors and Debugging

An error is a mistake in source code. A bug is a defect that causes incorrect or unexpected behavior. Debugging is the systematic process of finding, analyzing, and removing such defects.

Error Type	Meaning	Typical Example
Syntax	Violates C grammar	Missing semicolon
Compilation/Type	Invalid type or declaration	Incompatible assignment
Linker	Required definition cannot be linked	Undefined function
Runtime	Fails during execution	Division by zero / invalid memory
Logical	Runs but gives wrong result	Incorrect loop condition

Case 1: Syntax & Compilation Error

A missing semicolon prevents successful compilation. Clang identifies the source line and reports that a semicolon is expected.



```
UW PIC0 5.09 File: sample1.c Modified
#include<stdio.h>
int main()
{
  int a = 10
  printf("%d\n",a);
  return 0;
}
```

Get Help WriteOut Read File Prev Pg Cut Text Cur Pos
Exit Justify Where is Next Pg UnCut Text To Spell

Source code containing the syntax error.

```
dhruvpandey@DHRLVs-MacBook-Air ~ % clang sample1.c -o sample1.c
sample1.c:4:11: error: expected ';' at end of declaration
4 | int a = 10
  |         ^
  |         ;
1 error generated.
dhruvpandey@DHRLVs-MacBook-Air ~ %
```

Terminal output showing the Clang syntax-error diagnostic.

Case 2: Compiler Warnings (-Wall, -Wextra)

Warnings identify suspicious code that is still legal C. Here, variable a is declared but never used.



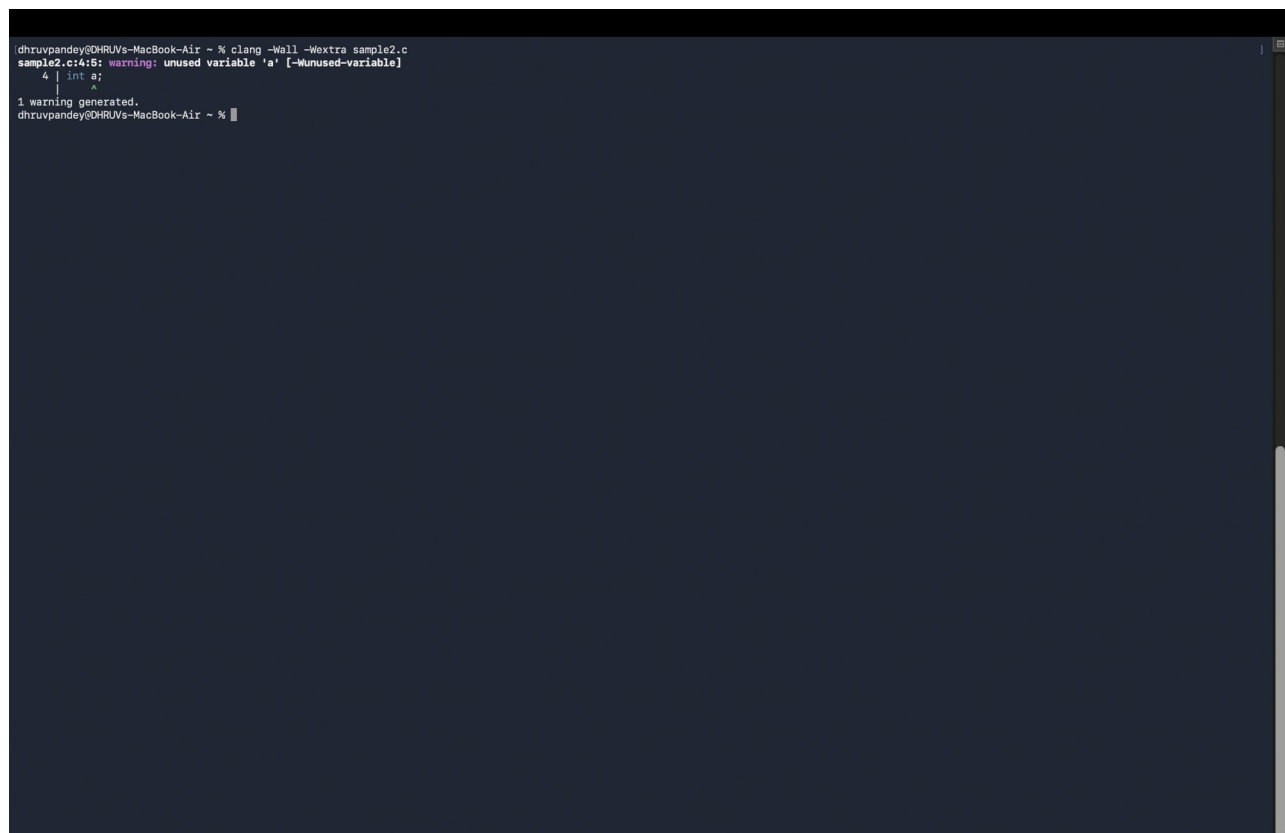
The screenshot shows a code editor window titled "UW PICO 5.09" with a file named "sample2.c". The code is as follows:

```
#include<stdio.h>
int main()
{
    int a;
    printf("Hello\n");

    return 0;
}
```

The editor has a dark theme and a menu bar at the bottom with various options like "Get Help", "Exit", "WriteOut", "Justify", "Read File", "Where is", "Prev Pg", "Next Pg", "Cut Text", "UnCut Text", "Cur Pos", and "To Spell".

Program containing an unused variable.



The screenshot shows a terminal window with the following output:

```
dhrupandey@DHRUVs-MacBook-Air ~ % clang -Wall -Wextra sample2.c
sample2.c:4:5: warning: unused variable 'a' [-Wunused-variable]
    4 | int a;
      |     ^
1 warning generated.
dhrupandey@DHRUVs-MacBook-Air ~ %
```

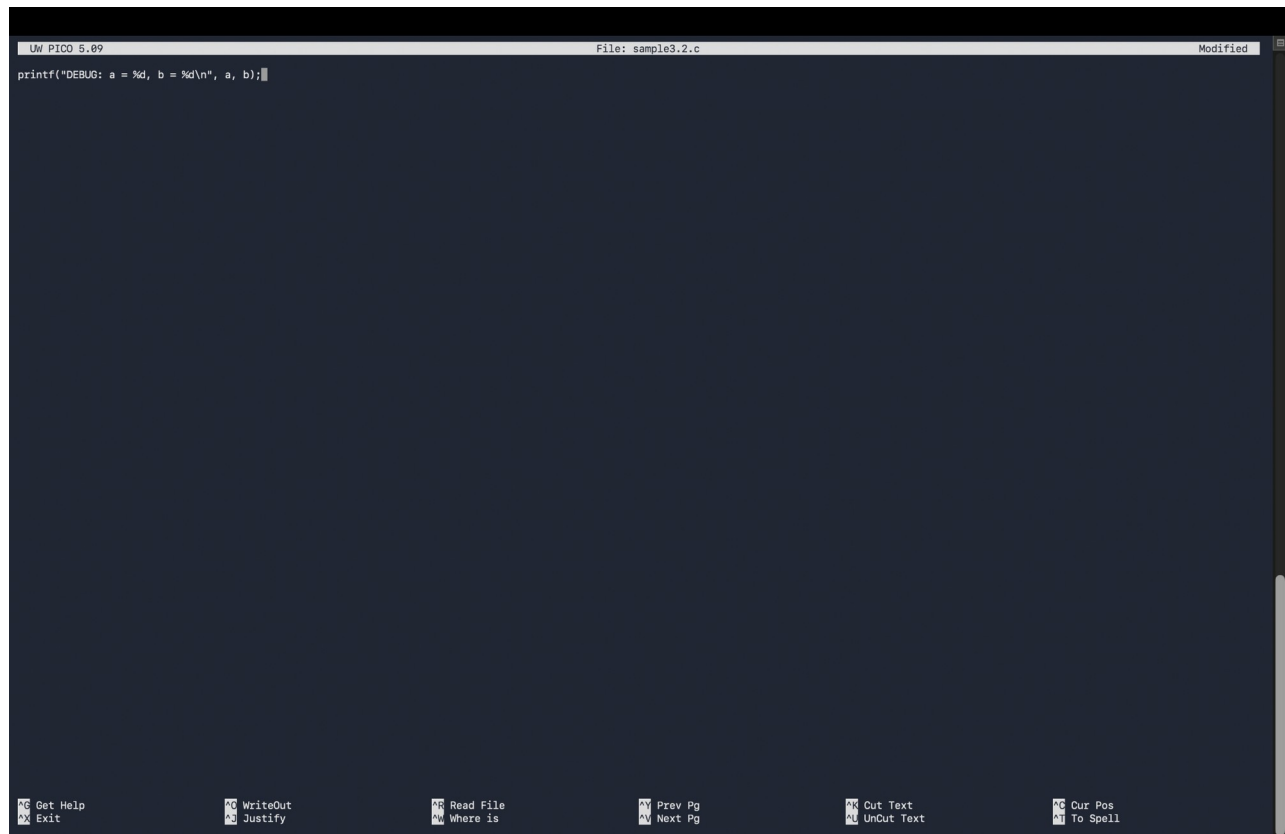
Terminal warning generated with -Wall and -Wextra.

Case 3: printf-Based Debugging (Logical Bug)

Debug print statements expose the values of `i` and `sum` on each loop iteration. The trace shows the program reaches a final sum of 10.

```
dhruvpandey@DHruVs-MacBook-Air ~ % clang sample3.c -o sample3
dhruvpandey@DHruVs-MacBook-Air ~ % ./sample3
DEBUG: i = 0
DEBUG: sum = 0
DEBUG: i = 1
DEBUG: sum = 1
DEBUG: i = 2
DEBUG: sum = 3
DEBUG: i = 3
DEBUG: sum = 6
DEBUG: i = 4
DEBUG: sum = 10
dhruvpandey@DHruVs-MacBook-Air ~ %
```

Terminal trace produced by printf-based debugging.



The image shows a screenshot of a text editor window. The title bar at the top reads "UW PICO 5.09" on the left, "File: sample3.2.c" in the center, and "Modified" on the right. The main editing area is dark blue and contains a single line of C code: `printf("DEBUG: a = %d, b = %d\n", a, b);`. The cursor is positioned at the end of this line. At the bottom of the window, there is a horizontal toolbar with several icons and labels: "Get Help", "Exit", "WriteOut", "Justify", "Read File", "Where is", "Prev Pg", "Next Pg", "Cut Text", "UnCut Text", "Cur Pos", and "To Spell".

Example of a debug printf statement.

Case 4: Debugging a Logical Error using GDB

The loop uses $i < 5$, so 5 is never included. GDB can stop on the sum statement and display i and sum at each iteration. The actual result is 10; the intended result is 15.

```
main.c
1 #include <stdio.h> // Line 1
2 int main() // Line 2
3 { // Line 3
4   int i, sum = 0; // Line 4
5   for(i = 1; i < 5; i++) // Line 5 (Logical Bug: should be i <= 5)
6   { // Line 6
7     sum = sum + i; // Line 7
8   } // Line 8
9   printf("Sum = %d\n", sum); // Line 9
10  return 0; // Line 10
11 }
```

Input

Debug Console

```
Reading symbols from a.out...
(gdb) break 7
Breakpoint 1 at 0x1151: file main.c, line 7.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:7
7   sum = sum + i; // Line 7
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at main.c:7
7   sum = sum + i; // Line 7
1: i = 2
2: sum = 1
```

```
main.c
1 #include <stdio.h> // Line 1
2 int main() // Line 2
3 { // Line 3
4   int i, sum = 0; // Line 4
5   for(i = 1; i < 5; i++) // Line 5 (Logical Bug: should be i <= 5)
6   { // Line 6
7     sum = sum + i; // Line 7
8   } // Line 8
9   printf("Sum = %d\n", sum); // Line 9
10  return 0; // Line 10
11 }
```

Input

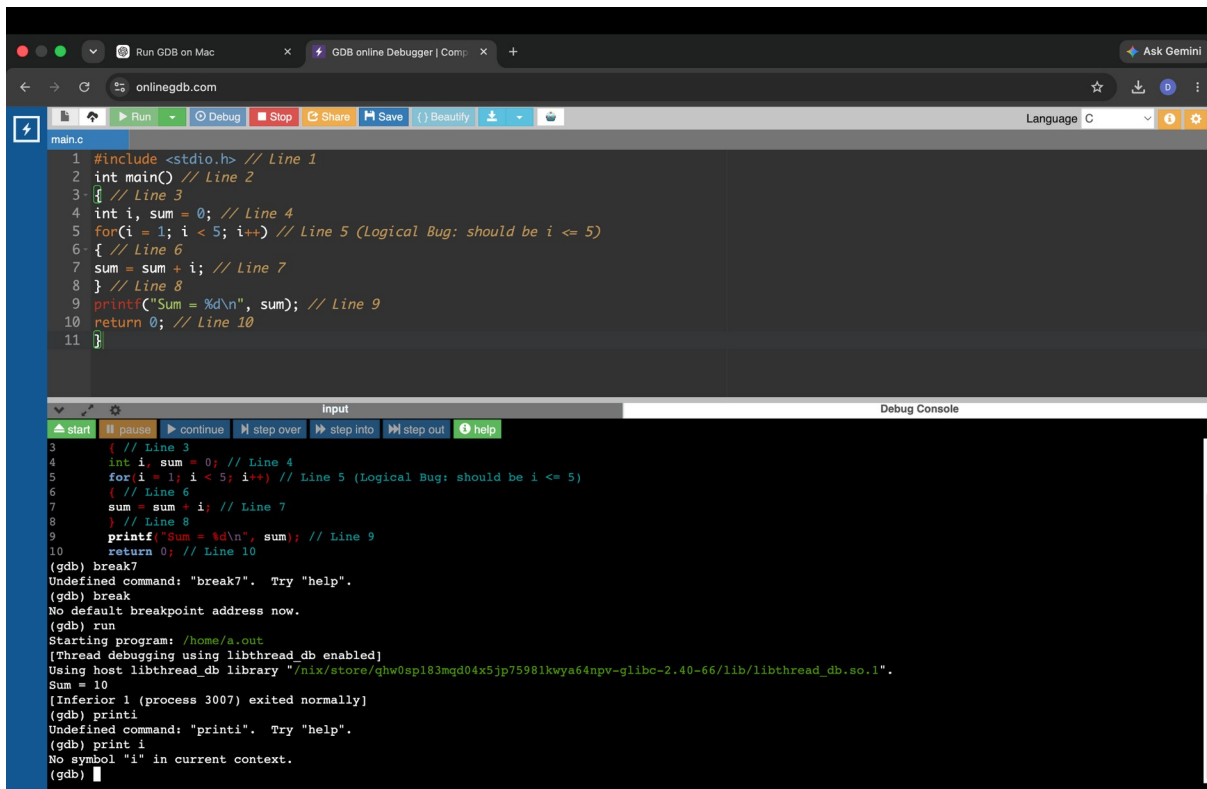
Debug Console

```
Breakpoint 1, main () at main.c:7
7   sum = sum + i; // Line 7
1: i = 2
2: sum = 1
(gdb) continue
Undefined command: "continue". Try "help".
(gdb) continue
Continuing.

Breakpoint 1, main () at main.c:7
7   sum = sum + i; // Line 7
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at main.c:7
7   sum = sum + i; // Line 7
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum = 10
```

GDB observation: when i reaches 5, the loop condition becomes false before the body executes. Therefore 5 is omitted from the sum.



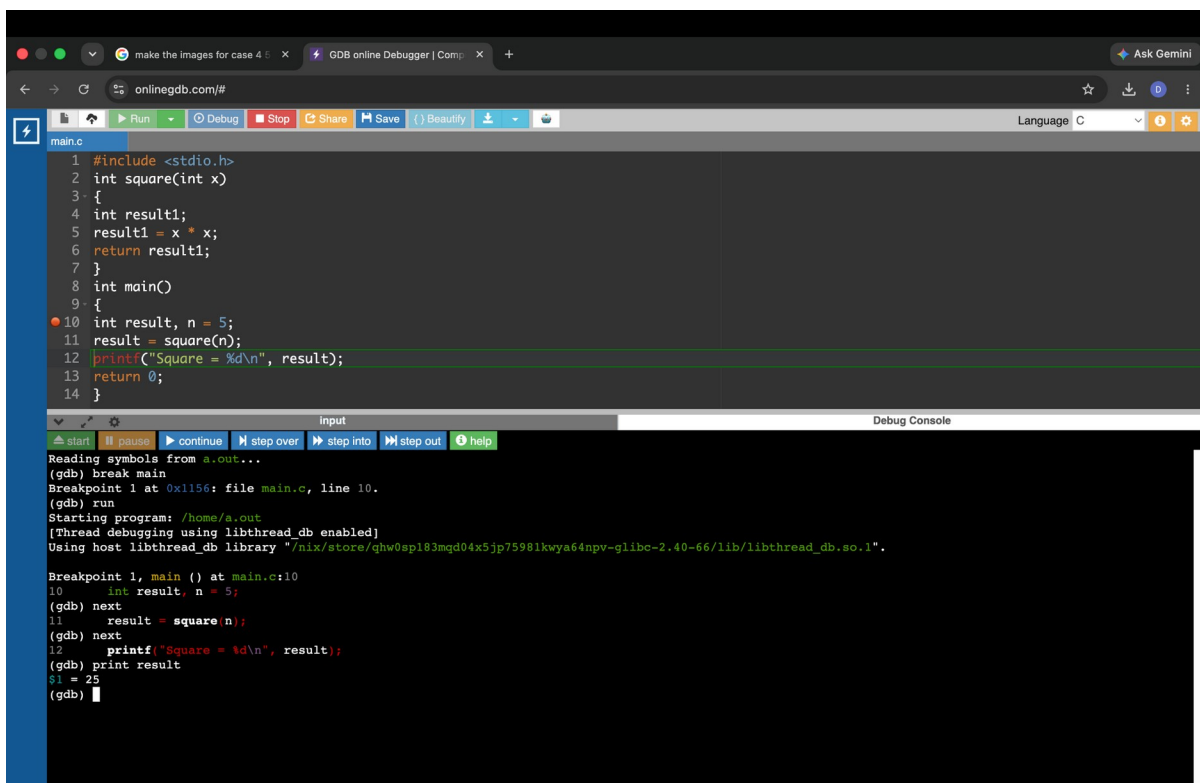
```
1 #include <stdio.h> // Line 1
2 int main() // Line 2
3 { // Line 3
4     int i, sum = 0; // Line 4
5     for(i = 1; i < 5; i++) // Line 5 (Logical Bug: should be i <= 5)
6     { // Line 6
7         sum = sum + i; // Line 7
8     } // Line 8
9     printf("Sum = %d\n", sum); // Line 9
10    return 0; // Line 10
11 }
```

```
(gdb) break 7
Undefined command: "break7". Try "help".
(gdb) break
No default breakpoint address now.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/ghw0sp183mqd04x5jp75981kwy64npv-glibc-2.40-66/lib/libthread_db.so.1".
Sum = 10
[Inferior 1 (process 3007) exited normally]
(gdb) print i
Undefined command: "printi". Try "help".
(gdb) print i
No symbol "i" in current context.
(gdb)
```

OnlineGDB logical-error debugging session.

Case 5: Debugging Functions (next vs. step)

GDB next executes a function call without entering it, while step enters the called function so its statements and local values can be inspected.



```
1 #include <stdio.h>
2 int square(int x)
3 {
4     int result1;
5     result1 = x * x;
6     return result1;
7 }
8 int main()
9 {
10    int result, n = 5;
11    result = square(n);
12    printf("Square = %d\n", result);
13    return 0;
14 }
```

```
Reading symbols from a.out...
(gdb) break main
Breakpoint 1 at 0x1156: file main.c, line 10.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/ghw0sp183mqd04x5jp75981kwy64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:10
10     int result, n = 5;
(gdb) next
11     result = square(n);
(gdb) next
12     printf("Square = %d\n", result);
(gdb) print result
$1 = 25
(gdb)
```


The screenshot shows the GDB online debugger interface. The code editor displays a C program named `main.c` with the following content:

```
1 #include <stdio.h>
2 int square(int x)
3 {
4     int result1;
5     result1 = x * x;
6     return result1;
7 }
8 int main()
9 {
10  int result, n = 5;
11  result = square(n);
12  printf("Square = %d\n", result);
13  return 0;
14 }
15
```

The debugger console shows the following output:

```
Reading symbols from a.out...
(gdb) break main
Breakpoint 1 at 0x1156: file main.c, line 10.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:10
10      int result, n = 5;
(gdb) next
11      result = square(n);
(gdb) step
square (x=5) at main.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
```

The screenshot shows the GDB online debugger interface. The code editor displays the same C program as the first screenshot. The debugger console shows the following output:

```
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:10
10      int result, n = 5;
(gdb) next
11      result = square(n);
(gdb) step
square (x=5) at main.c:5
5      result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6      return result1;
(gdb) print result1
$2 = 25
(gdb)
```

Case 6: Runtime Error using GDB Backtrace

A division by zero produces SIGFPE. The backtrace command shows the chain divide -> calculate -> main, making it possible to locate the origin of the crash.

The screenshot shows the GDB online debugger interface. The code in `main.c` defines a `divide` function that returns `a / b` and a `calculate` function that calls `divide(x, 0)`. The `main` function calls `calculate(10)` and prints the result. The debugger has executed the program, and a SIGFPE (Arithmetic exception) signal was received. The backtrace shows the error occurred in `divide` at `main.c:4`. The register `$1` contains the value `0`.

```
main.c
1 #include <stdio.h>
2 int divide(int a, int b)
3 {
4     return a / b; // Division by zero when b = 0
5 }
6 int calculate(int x)
7 {
8     return divide(x, 0); // Passes 0 as divisor
9 }
10 int main()
11 {
12     int result = calculate(10);
13     printf("Result: %d\n", result);
14     return 0;
15 }
```

Input:
Debug Console:
Reading symbols from a.out...
(gdb) break
No default breakpoint address now.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mgd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".
Program received signal SIGFPE, Arithmetic exception.
0x000055555555147 in divide (a=10, b=0) at main.c:4
4 return a / b; // Division by zero when b = 0
(gdb) backtrace
#0 0x000055555555147 in divide (a=10, b=0) at main.c:4
#1 0x000055555555166 in calculate (x=10) at main.c:8
#2 0x00005555555517a in main () at main.c:12
(gdb) frame 0
#0 0x000055555555147 in divide (a=10, b=0) at main.c:4
4 return a / b; // Division by zero when b = 0
(gdb) print b
\$1 = 0
(gdb) frame 1

The screenshot shows the GDB online debugger interface. The code in `main.c` defines a `divide` function that returns `a / b` and a `calculate` function that calls `divide(x, 0)`. The `main` function calls `calculate(10)` and prints the result. The debugger has executed the program, and a SIGFPE (Arithmetic exception) signal was received. The backtrace shows the error occurred in `divide` at `main.c:4`. The register `$1` contains the value `0`.

```
main.c
1 #include <stdio.h>
2 int divide(int a, int b)
3 {
4     return a / b; // Division by zero when b = 0
5 }
6 int calculate(int x)
7 {
8     return divide(x, 0); // Passes 0 as divisor
9 }
10 int main()
11 {
12     int result = calculate(10);
13     printf("Result: %d\n", result);
14     return 0;
15 }
```

Input:
Debug Console:
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mgd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".
Program received signal SIGFPE, Arithmetic exception.
0x000055555555147 in divide (a=10, b=0) at main.c:4
4 return a / b; // Division by zero when b = 0
(gdb) backtrace
#0 0x000055555555147 in divide (a=10, b=0) at main.c:4
#1 0x000055555555166 in calculate (x=10) at main.c:8
#2 0x00005555555517a in main () at main.c:12
(gdb) frame 0
#0 0x000055555555147 in divide (a=10, b=0) at main.c:4
4 return a / b; // Division by zero when b = 0
(gdb) print b
\$1 = 0
(gdb) frame 1
#1 0x000055555555166 in calculate (x=10) at main.c:8
8 return divide(x, 0); // Passes 0 as divisor
(gdb) print x
\$2 = 10
(gdb)

Case 7: Array Out-of-Bounds Error using GDB

An array of five integers has valid indices 0 through 4. The condition `i <= 5` also accesses `a[5]`, which is outside the array and causes undefined behavior.

The screenshot shows the GDB online debugger interface. The source code in `main.c` is as follows:

```
1 #include <stdio.h>
2 int main()
3 {
4     int a[5];
5     int i;
6     for(i = 0; i <= 5; i++)
7     {
8         scanf("%d", &a[i]); //Line 8
9     }
10    return 0;
11 }
```

A breakpoint is set at line 8. The program is run, and the debugger stops at the breakpoint. The debug console shows the following output:

```
(gdb) break 8
Breakpoint 1 at 0x114a: file main.c, line 8.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mgd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:8
8     scanf("%d", &a[i]); //Line 8
(gdb) print &a[0]
Undefined command: "print". Try "help".
(gdb) print &a[0]
$1 = (int *) 0x7fffffff60
(gdb) print &a[4]
$2 = (int *) 0x7fffffff70
(gdb) print &a[5]
$3 = (int *) 0x7fffffff74
(gdb)
```

The program crashes with a segmentation fault (SIGSEGV) because it attempts to access memory at address 0x7fffffff74, which is outside the bounds of the array `a`.

Case 8: Segmentation Fault (SIGSEGV) using GDB

The pointer `p` is NULL (address 0x0). Writing through it with `*p = 10` attempts to access invalid memory, so the program receives SIGSEGV.

The screenshot shows the GDB online debugger interface. The source code in `main.c` is as follows:

```
1 #include <stdio.h>
2 int main()
3 {
4     int *p = NULL; // Pointer points to address 0x0
5     *p = 10; // Attempting to write to address 0x0 causes Segmentation Fault
6     printf("%d\n", *p);
7     return 0;
8 }
```

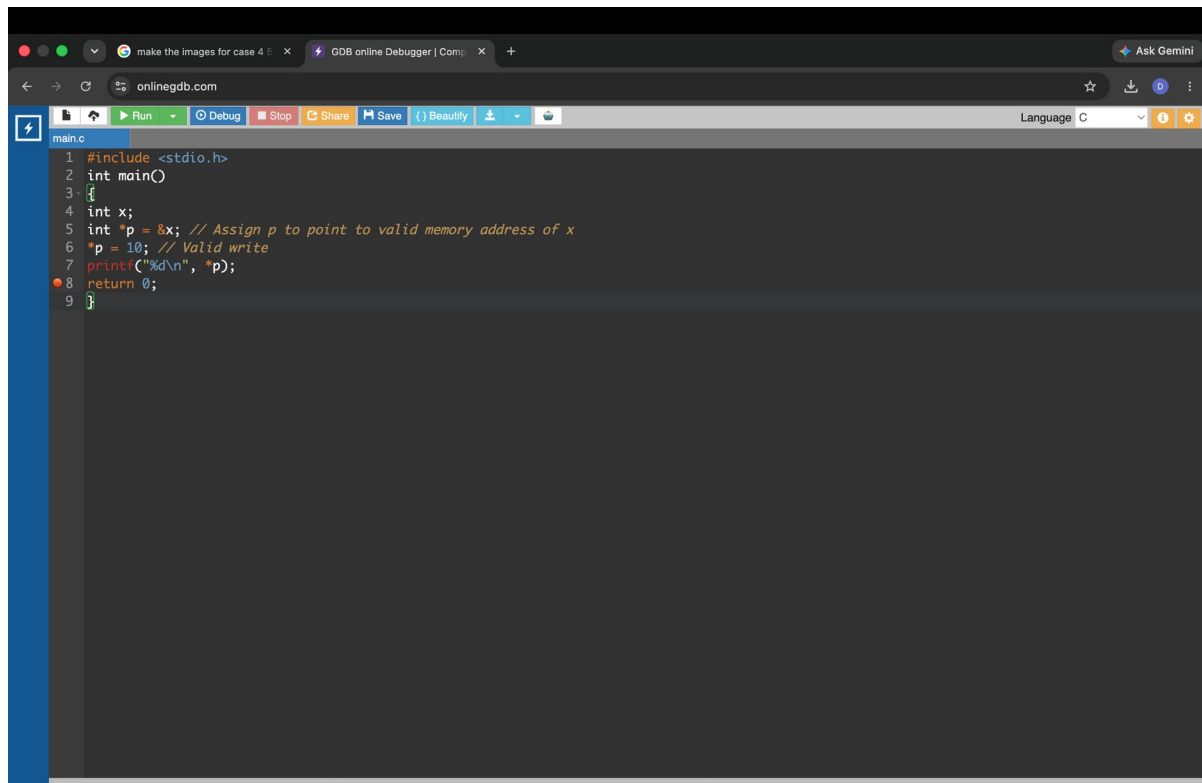
A breakpoint is set at line 8. The program is run, and the debugger stops at the breakpoint. The debug console shows the following output:

```
(gdb) info break
Num      Type      Disp Enb Address      What
1        breakpoint keep y  0x00000000000001174 in main at main.c:8

(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mgd04x5jp75981kwya64npv-glibc-2.40-66/lib/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x0000555555555514d in main () at main.c:5
5     *p = 10; // Attempting to write to address 0x0 causes Segmentation Fault
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
Cannot access memory at address 0x0
(gdb)
```

The program crashes with a segmentation fault (SIGSEGV) because it attempts to write to memory at address 0x0, which is invalid.



The screenshot shows a web browser window with the URL `onlinegdb.com`. The page title is "GDB online Debugger | Comp". The interface includes a toolbar with buttons for Run, Debug, Stop, Share, Save, and Beautify. The code editor displays a C program named `main.c` with the following content:

```
1 #include <stdio.h>
2 int main()
3 {
4     int x;
5     int *p = &x; // Assign p to point to valid memory address of x
6     *p = 10; // Valid write
7     printf("%d\n", *p);
8     return 0;
9 }
```

Systematic Debugging Strategy

1. Observe expected vs. actual output
2. Reproduce the issue consistently
3. Compile with `-Wall -Wextra -g`
4. Isolate suspicious code
5. Inspect state using `printf` or GDB
6. Apply one minimal fix
7. Recompile and retest
8. Verify edge cases and boundary inputs

Conclusion

Compiler diagnostics, warning flags, `printf` tracing, and GDB provide complementary ways to locate errors. Breakpoints and variable inspection help with logical bugs; next and step help analyze functions; backtrace identifies crash call chains; and pointer/array inspection helps identify invalid memory access.